```
import numpy as np


class GridWorldEnv:
    def __init__(self):
        self.grid_shape = (2, 3)
        self.states = {
            'S0': (0, 0), 'S1': (0, 1), 'G': (0, 2),
            'X': (1, 0), 'S3': (1, 1), 'S4': (1, 2)
        }
        self.terminal_states = {'G'}
        self.obstacle_states = {'X'}
        self.gamma = 0.9

    def get_next_state_and_reward(self, state, action):
        if state in self.terminal_states:
            return state, 0, True

        r, c = self.states[state]

        # Action movements
        if action == "Up":
            next_r, next_c = r - 1, c
        elif action == "Down":
            next_r, next_c = r + 1, c
        elif action == "Left":
            next_r, next_c = r, c - 1
        elif action == "Right":
            next_r, next_c = r, c + 1
        else:
            raise ValueError("Invalid Action")


        if next_r < 0 or next_r >= 2 or next_c < 0 or next_c >= 3:
            next_r, next_c = r, c


        next_state = [k for k, v in self.states.items() if v == (next_r, next_c)][0]


        if next_state == 'G':
            reward = 10
            is_terminal = True
        elif next_state == 'X':
            reward = -10

            next_state = state
            is_terminal = False
        else:
```

```
            reward = 0
            is_terminal = False

        return next_state, reward, is_terminal


policy = {
    'S0': 'Right',
    'S1': 'Down',
    'S3': 'Right',
    'S4': 'Up'
}

env = GridWorldEnv()

def state_value_function(start_state, env, policy, gamma=0.9):
    """Calculates state value by following path without Bellman equation"""
    current_state = start_state
    discounted_return = 0.0
    discount_factor = 1.0

    while current_state not in env.terminal_states:
        action = policy[current_state]
        next_state, reward, is_terminal = env.get_next_state_and_reward(current_state, action)

        discounted_return += discount_factor * reward
        discount_factor *= gamma
        current_state = next_state

        if is_terminal:
            break

    return discounted_return

def action_value_function(start_state, first_action, env, policy, gamma=0.9):
    """Calculates action value for taking an initial action then following policy"""
    current_state = start_state
    discounted_return = 0.0
    discount_factor = 1.0

    # First action step
    next_state, reward, is_terminal = env.get_next_state_and_reward(current_state, first_action)
    discounted_return += discount_factor * reward
    discount_factor *= gamma
    current_state = next_state

    # Subsequent steps following policy
    while current_state not in env.terminal_states and not is_terminal:
        action = policy[current_state]
        next_state, reward, is_terminal = env.get_next_state_and_reward(current_state, action)

        discounted_return += discount_factor * reward
```

```
        discount_factor *= gamma
        current_state = next_state

    return discounted_return

# Output Execution
if __name__ == "__main__":
    v_s0 = state_value_function('S0', env, policy, gamma=0.9)
    q_s0_up = action_value_function('S0', 'Up', env, policy, gamma=0.9)

    print(f"State Value V(S0): {v_s0}")
    print(f"Action Value Q(S0, 'Up'): {q_s0_up}")
```

```
State Value V(S0): 7.290000000000001
Action Value Q(S0, 'Up'): 6.561000000000002
```